GUÍA DEFINITIVA
PARA LA PARTE PRÁCTICA

Despliegue de Aplicaciones Web

Orden exacto de trabajo · clics · rutas · comprobaciones · errores reales

ENTORNO YA PROBADO: Windows 11 · JDK 25.0.3 · Apache Tomcat 9.0.123 · Eclipse Enterprise · puerto Tomcat 8080

IDEA CLAVE: Primero haz que funcione en Eclipse. Después exporta el WAR. Solo después prueba el WAR fuera de Eclipse.

Esta guía está ordenada para seguirla durante el examen.
Los bloques marcados como “SOLO SI…” se hacen únicamente cuando el enunciado lo pida.

0. ORDEN GENERAL DEL EXAMEN

NO EMPIECES TOCANDO COSAS: Lee el enunciado completo y marca: qué archivo te dan, qué debes desplegar, si hay Servlet/JSP, si piden WAR y qué URL final debe funcionar.

Paso	Qué haces	Objetivo
1	Leer el enunciado	Identificar ZIP / proyecto Eclipse / WAR / Servlet-JSP / URL / entrega.
2	Comprobar Java y puerto 8080	java -version · javac -version · netstat.
3	Comprobar Tomcat	Usar Eclipse si el ejercicio se trabaja desde Eclipse. Standalone solo cuando corresponda.
4	Preparar el proyecto	Crear Dynamic Web Project o importar el proyecto entregado.
5	Integrar la web	HTML/CSS/JS dentro de src/main/webapp.
6	Primer despliegue	Run As → Run on Server → comprobar web antes de añadir lógica.
7	Añadir Servlet/JSP	Solo si el ejercicio lo pide.
8	Probar formulario/Servlet	Publicar/reiniciar si un Servlet nuevo da 404.
9	Exportar WAR	Solo cuando el proyecto ya funciona.
10	Despliegue independiente	Solo si piden probar el WAR fuera de Eclipse.
11	Comprobación final	Recorrer páginas, JS, JSP, Servlet, rutas y URL exigida.

REGLA DE ORO: No tengas Tomcat standalone y Tomcat de Eclipse arrancados a la vez si ambos usan el puerto 8080.

1. PASOS QUE HACES SIEMPRE AL EMPEZAR

1.1. Lee el enunciado entero

- Subraya el nombre que debe tener el proyecto o WAR.
- Mira si te dan un ZIP con HTML/CSS/JS, un proyecto de Eclipse o un WAR.
- Mira si piden Servlet/JSP o solo despliegue.
- Mira si piden exportar WAR o desplegarlo manualmente.
- Anota la URL final que debe funcionar.

1.2. Comprueba Java

```
java -version
javac -version
```

RESULTADO ESPERADO: Ambos comandos responden. En tu equipo actual: JDK 25.0.3.

1.3. Comprueba el puerto 8080

```
netstat -ano | findstr :8080
```

- Si no aparece nada: 8080 está libre.
- Si aparece LISTENING: hay un proceso usando 8080. No arranques otro Tomcat todavía.

```
tasklist /FI "PID eq 1234"
```

SI EL 8080 ESTÁ OCUPADO: Identifica el PID y cierra el Tomcat/proceso que sobra. No cambies el puerto sin necesidad.

2. TOMCAT: QUÉ ARRANCAR Y CUÁNDO

2.1. Si vas a trabajar desde Eclipse

- No arranques primero catalina.bat run.
- Abre Eclipse y usa el Tomcat de la vista Servers.
- Si el Tomcat standalone estaba abierto, deténlo con Ctrl+C antes.

2.2. Comprobar Tomcat standalone (solo si el enunciado lo pide o necesitas verificar el servidor)

```
cd C:\Users\daw2\Downloads\apache-tomcat-9.0.123-windows-x64\apache-tomcat-9.0.123\bin
catalina.bat run
```

- Prueba http://localhost:8080/.
- Para detenerlo: Ctrl+C.

IMPORTANTE: Cuando Tomcat lo gestiona Eclipse, entrar en http://localhost:8080/ puede devolver 404 aunque el servidor esté funcionando. Comprueba el estado [Started, Synchronized] y la URL de tu proyecto.

2.3. Si aparece JAVA_HOME no definido

```
TOMCAT\bin\setenv.bat

set "JAVA_HOME=C:\Program Files\Java\jdk-25.0.3"
```

3. ABRIR ECLIPSE Y COMPROBAR EL SERVIDOR

3.1. Abre el workspace

```
C:\Users\daw2\eclipse-workspace
```

3.2. Comprueba la vista Servers

1. Window → Show View → Servers (solo si no ves la pestaña Servers).
2. Comprueba que existe “Tomcat v9.0 Server at localhost”.
3. Si pone [Stopped], clic derecho → Start.
4. Espera a [Started, Synchronized].

SI YA EXISTE EL SERVIDOR: No crees otro Tomcat duplicado. Usa el que ya está configurado.

3.3. Si NO existe el Runtime de Tomcat (solo si falta)

5. Window → Preferences → Server → Runtime Environments.
6. Add… → Apache → Apache Tomcat v9.0.
7. Tomcat installation directory → carpeta raíz de Tomcat 9.0.123.
8. Selecciona el JDK válido → Finish.
9. Servers → New → Server → Tomcat v9.0 Server → Finish.

4. PREPARAR EL PROYECTO SEGÚN LO QUE TE DEN

ELIGE SOLO UNA RUTA: ZIP de web → 4A. Proyecto Eclipse existente → 4B. WAR ya compilado → 4C.

4A. SI TE DAN UN ZIP CON HTML/CSS/JS

10. Descomprime el ZIP en una carpeta fácil de localizar.
11. Eclipse → File → New → Dynamic Web Project.
12. Project name → escribe exactamente el nombre pedido.
13. Target runtime → Apache Tomcat v9.0.
14. Deja el resto por defecto → Finish.
15. En Project Explorer abre: proyecto → src → main → webapp.
16. Copia DENTRO de webapp el contenido del ZIP: HTML, CSS, JS, imágenes, etc.
17. Si Eclipse no los muestra: clic derecho en el proyecto → Refresh.

```
src/main/webapp/
├── index.html
├── productos.html
├── contacto.html
├── css/
├── js/
├── img/
├── META-INF/
└── WEB-INF/
```

4B. SI TE DAN UN PROYECTO DE ECLIPSE

18. Descomprime el proyecto si viene en ZIP.
19. File → Import… → General → Existing Projects into Workspace → Next.
20. Select root directory → selecciona la carpeta que contiene .project.
21. Marca el proyecto → Finish.
22. Clic derecho proyecto → Properties → Targeted Runtimes.
23. Marca Apache Tomcat v9.0 si no está marcado → Apply and Close.
24. Comprueba que no haya errores rojos.

4C. SI TE DAN DIRECTAMENTE UN WAR

- No necesitas crear un Dynamic Web Project para desplegarlo manualmente.
- Ve directamente a la sección 9: despliegue independiente del WAR.

5. PRIMER DESPLIEGUE DESDE ECLIPSE

5.1. Ejecuta el proyecto

25. Clic derecho sobre el proyecto.
26. Run As → Run on Server.
27. Selecciona Tomcat v9.0 Server at localhost.
28. Finish.
29. Si Eclipse pregunta “The server may need be restarted” → selecciona Restart server → OK.
30. Espera a [Started, Synchronized].

5.2. Prueba la URL

```
http://localhost:8080/NOMBRE_PROYECTO/
```

- Comprueba index.html o index.jsp.
- Abre todas las páginas.
- Comprueba CSS, imágenes y JavaScript.
- Si hay un botón JS, pruébalo.

NO SIGAS SI ESTO FALLA: Antes de crear Servlets, consigue que la web básica cargue correctamente.

6. SOLO SI PIDEN SERVLET/JSP: CREAR EL SERVLET

6.1. Crear Servlet desde Eclipse

31. Clic derecho en el proyecto → New → Servlet. Si no aparece: New → Other… → Web → Servlet.
32. Java package → escribe el paquete pedido o uno sencillo, por ejemplo com.ejemplo.
33. Class name → nombre pedido, por ejemplo ConsultaServlet.
34. Comprueba superclass: javax.servlet.http.HttpServlet.
35. Next >.
36. Comprueba URL mappings: /ConsultaServlet.
37. Next >.
38. Deja marcados doGet y doPost (doPost es imprescindible si el formulario usa POST).
39. Finish.

TOMCAT 9: Usa javax.servlet. Si ves jakarta.servlet en código copiado de otro sitio, no es el patrón que habéis trabajado con Tomcat 9.

6.2. Plantilla mínima de doPost

```
import java.io.PrintWriter;

protected void doPost(HttpServletRequest request, HttpServletResponse response)
      throws ServletException, IOException {

   request.setCharacterEncoding("UTF-8");

   String nombre = request.getParameter("nombre");
   String producto = request.getParameter("producto");

   response.setContentType("text/html;charset=UTF-8");
   PrintWriter out = response.getWriter();

   out.println("<h1>Consulta recibida</h1>");
   out.println("<p>Gracias, " + nombre + ".</p>");
   out.println("<p>Producto: " + producto + ".</p>");
   out.println("<a href='consulta.jsp'>Volver</a>");
}
```

CLAVE: El texto de request.getParameter("...") debe coincidir EXACTAMENTE con el atributo name del formulario.

7. SOLO SI PIDEN FORMULARIO/JSP: CREAR LA JSP

7.1. Crear consulta.jsp

40. Clic derecho en src/main/webapp → New → JSP File.
41. File name → consulta.jsp → Finish.
42. Sustituye la plantilla por el formulario pedido.

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
   pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html lang="es">
<head>
   <meta charset="UTF-8">
   <title>Consulta</title>
</head>
<body>
   <form method="post" action="ConsultaServlet">
      <label>Nombre:</label>
      <input type="text" name="nombre" required>

      <label>Producto:</label>
      <input type="text" name="producto" required>

      <button type="submit">Enviar</button>
   </form>
</body>
</html>
```

7.2. Enlazar la JSP desde la web (si lo pide)

```
<a href="consulta.jsp">Hacer una consulta</a>
```

8. PROBAR JSP + SERVLET

8.1. Guardar y publicar

43. Ctrl + S en los archivos modificados.
44. Mira la vista Servers.
45. Si aparece Republish → clic derecho en Tomcat → Publish.
46. Si acabas de crear un Servlet y da 404 → clic derecho en Tomcat → Restart.
47. Espera otra vez a [Started, Synchronized].

8.2. Prueba el flujo completo

```
http://localhost:8080/NOMBRE_PROYECTO/consulta.jsp
```

- Rellena el formulario.
- Pulsa Enviar.
- Comprueba que la URL termina en /ConsultaServlet.
- Comprueba que aparecen los datos introducidos.
- Prueba tildes y ñ. Si salen caracteres raros, revisa UTF-8.

ERROR REAL DEL SIMULACRO: Si /ConsultaServlet dio 404 justo después de crearlo, reiniciar Tomcat desde Eclipse solucionó la publicación del Servlet.

9. EXPORTAR EL WAR (CUANDO TODO FUNCIONE)

48. Clic derecho sobre el proyecto.
49. Export…
50. Web → WAR file → Next.
51. Web project → comprueba el proyecto correcto.
52. Destination → elige una ruta fácil, por ejemplo C:\Users\daw2\Desktop\MiProyecto.war.
53. Finish.
54. Comprueba que el archivo .war existe.

NO EXPORTES DEMASIADO PRONTO: Primero verifica HTML/CSS/JS + JSP + Servlet. El WAR debe ser el resultado final de un proyecto que ya funciona.

10. SOLO SI PIDEN DESPLIEGUE INDEPENDIENTE DEL WAR

10.1. Detener Tomcat de Eclipse

55. Vista Servers → clic derecho en Tomcat → Stop.
56. Espera a [Stopped].

10.2. Copiar WAR a webapps

```
C:\Users\daw2\Downloads\apache-tomcat-9.0.123-windows-x64\apache-tomcat-9.0.123\webapps
```

10.3. Arrancar Tomcat standalone

```
cd C:\Users\daw2\Downloads\apache-tomcat-9.0.123-windows-x64\apache-tomcat-9.0.123\bin
catalina.bat run
```

10.4. Probar la aplicación desplegada

```
http://localhost:8080/NOMBRE_WAR/
```

- Comprueba portada y navegación.
- Comprueba CSS/JS/imágenes.
- Comprueba consulta.jsp.
- Envía el formulario y verifica el Servlet.

CONTEXTO: Normalmente MiProyecto.war se visita como /MiProyecto/. Si cambias el nombre del WAR, cambia también la URL.

11. COMPROBACIÓN FINAL ANTES DE ENTREGAR

☐ Java y javac responden.
☐ Solo hay un Tomcat usando 8080.
☐ La URL exigida por el enunciado carga.
☐ index.html / index.jsp carga.
☐ Todas las páginas enlazan correctamente.
☐ CSS se aplica.
☐ JavaScript funciona.
☐ Imágenes cargan.
☐ JSP carga.
☐ Formulario envía por el método pedido.
☐ Servlet responde.
☐ Los parámetros se reciben correctamente.
☐ Tildes y ñ se ven bien.
☐ WAR generado si lo piden.
☐ WAR probado fuera de Eclipse si lo piden.
☐ Archivos guardados en la ruta de entrega.

12. ERRORES RÁPIDOS Y QUÉ HACER

Problema	Solución rápida
Puerto 8080 ocupado	netstat -ano | findstr :8080 → identifica PID → cierra el Tomcat/proceso que sobra.
Tomcat: JAVA_HOME no definido	Revisa TOMCAT\bin\setenv.bat y la ruta del JDK.
Tomcat de Eclipse da 404 en /	Puede ser normal. Comprueba [Started, Synchronized] y prueba /NombreProyecto/.
Proyecto da 404	Comprueba nombre/contexto, Run on Server y que el proyecto esté añadido al servidor.
Servlet nuevo da 404	Guardar → Publish → Restart Tomcat → volver a probar.
Imports javax.servlet en rojo	Properties → Targeted Runtimes → marcar Tomcat v9.0.
Código usa jakarta.servlet	Adaptar a javax.servlet para vuestro Tomcat 9.
CSS/JS no carga	Revisar que las carpetas estén dentro de src/main/webapp y que las rutas relativas sean correctas.
IvÃ¡n / caracteres raros	JSP en UTF-8 + request.setCharacterEncoding("UTF-8") antes de getParameter + response UTF-8.
Cambios no aparecen	Ctrl+S → Publish/Republish → si hace falta Restart.

WAR no funciona	Comprobar WAR en webapps, nombre/contexto, logs y que el Tomcat de Eclipse esté detenido.

13. MAPA MENTAL MÍNIMO

Elemento	Qué significa
src/main/webapp	Aquí van HTML, CSS, JS, imágenes y JSP.
src/main/java	Aquí van las clases Java y Servlets.
@WebServlet("/X")	Define la ruta del Servlet.
form action="X"	Indica a qué Servlet se envía el formulario.
method="post"	La petición llega a doPost().
request.getParameter("nombre")	Lee el campo cuyo name es "nombre".
WAR	Paquete desplegable de la aplicación web Java.
webapps	Carpeta de Tomcat para desplegar WARs manualmente.
localhost:8080	Dirección habitual de Tomcat en vuestro entorno.

14. VERSIÓN ULTRARRÁPIDA PARA MIRAR EN EL EXAMEN

1) Leer enunciado completo.
2) java -version / javac -version.
3) netstat -ano | findstr :8080.
4) Eclipse → Servers → Tomcat 9.
5) Crear/importar proyecto.
6) HTML/CSS/JS → src/main/webapp.
7) Run As → Run on Server.
8) Comprobar web básica.
9) Si piden Servlet: New → Servlet → javax.servlet → mapping → doPost.
10) Si piden JSP: New → JSP File → form POST → action del Servlet.
11) Guardar → Publish/Restart si hace falta.
12) Probar formulario y Servlet.
13) Export → Web → WAR file.
14) Si piden despliegue manual: Stop Eclipse Tomcat → WAR a webapps → catalina.bat run.
15) Probar URL final y TODO antes de entregar.

ÚLTIMA REGLA: Si algo falla, cambia una sola cosa cada vez. Primero puerto, luego servidor, luego proyecto/ruta, luego Servlet/JSP, y por último logs.

Alcance de la guía: basada en las prácticas realizadas (especialmente la Actividad 2), el ejemplo de Servlet proporcionado por el profesor y el simulacro completo realizado con ByteMarket. No incluye IIS ni XAMPP como ruta de examen.